

## Multi-Domain Cybersecurity Assessment (Practice Lab)

**Domains:** OSINT, Digital Forensics, Reverse Engineering

**Author:** Sailaxman Mudapaka

**Date:** Jan 2026

### Task 1: OSINT – Status Update

**Note:** All identities and repositories referenced in this task are part of a controlled, fictional assessment scenario.

### Objective:

This task's goal is to use OSINT techniques to look into publicly available information about a freelancer named Target Freelancer (Pseudonym), find any incidents, outages, or status updates connected to him, and identify any hidden flags related to the investigation.

### Scope & Ethics :

The research was done solely on publicly available data. There was no exploitation, scanning, or illegal access of a system or an asset. The ethical way to complete the task, as per the instructions, was followed.

### Tools Used:

- Web Browser
- DuckDuckGo Search Engine
- GitHub (Repository & Issue Tracker)

### Methodology:

The research work started with a keyword search employing the freelancer's name, which is Target Freelancer (Pseudonym), combined with related words such as “status update,” “outage,” and “incident.”

To accomplish the above process, a GitHub account that had the name ***Target GitHub Account*** attached to it was discovered. Based on the requirement of fulfilling the task narrative that incorporated a software project and a freelancer, the discovery of the GitHub account seemed appropriate for additional research.

The repositories related to this profile were examined. There was a repository named ***“Sample Organization Repository”*** that was closely related to the task narrative. An analysis of the contents, documentation, and issues related to the repository was performed to look for any information related to communication or status updates on the incident.

“The investigation was continued only on publicly visible artifacts to ensure compliance with ethical OSINT practices.”

Figure 1: Search engine results identifying GitHub repository

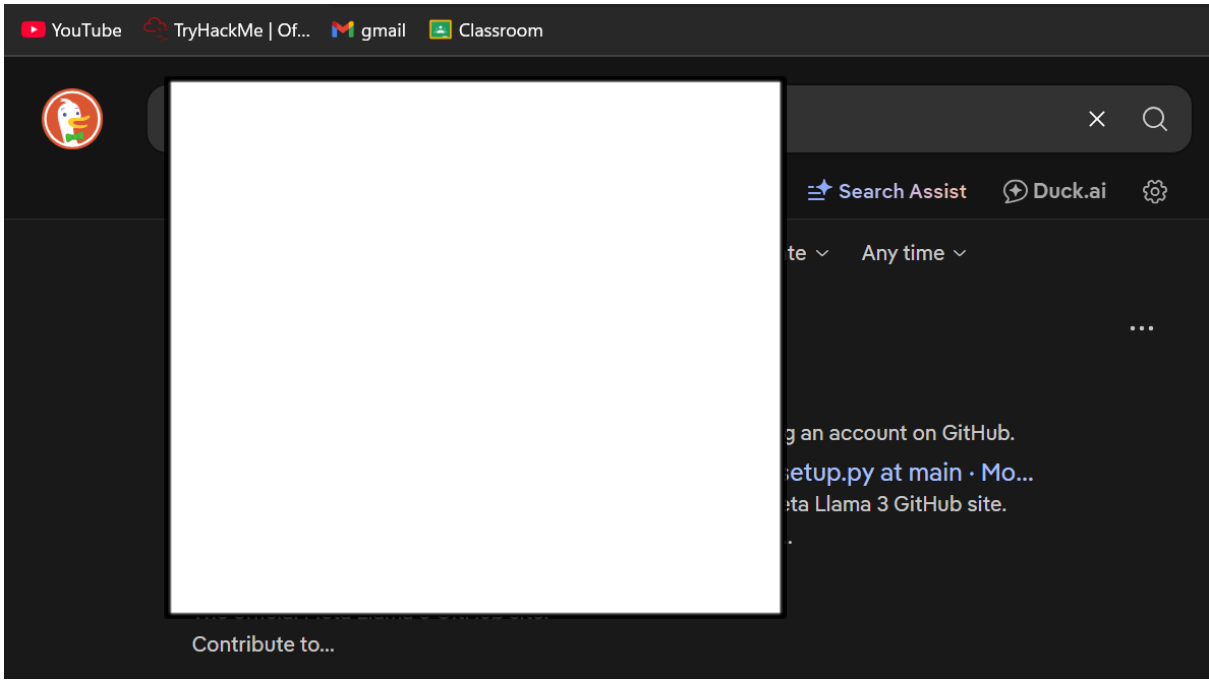
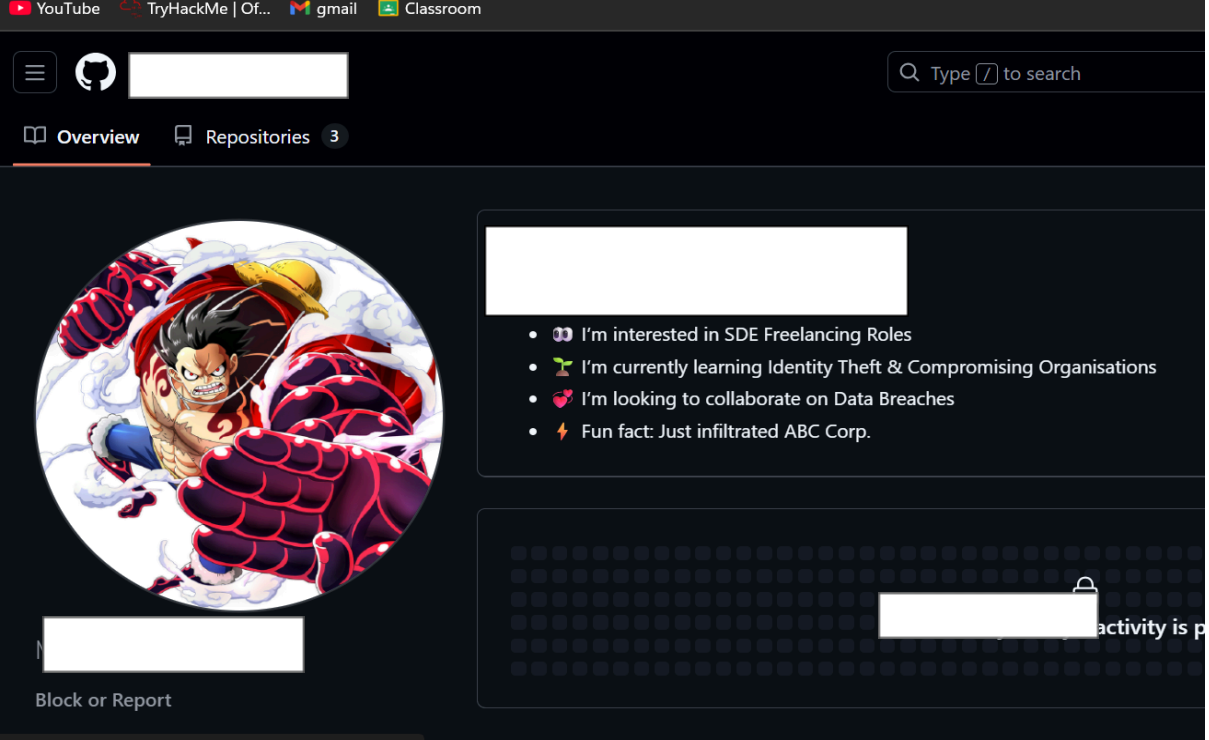
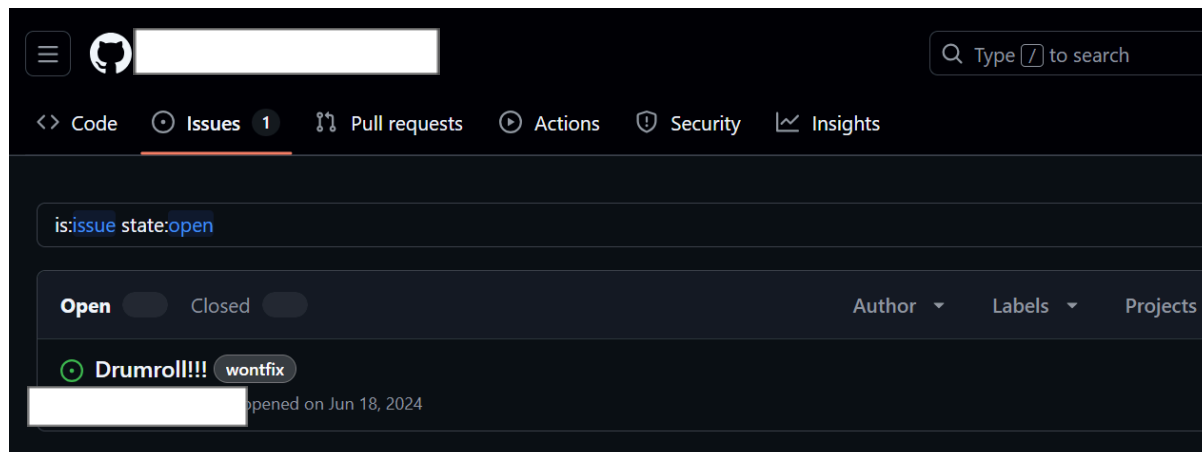


Figure 2 :GitHub Profile



**Figure 3: GitHub Issue containing the flag**



### **Relation to the Keywords:**

The keywords 'incident', 'status update', and 'outage' are to be considered as context pointers, rather than exact keywords.

- GitHub Issue is a representation of the incident that is captured while the software is under development.
- The issue description is a status update that provides information about the event.
- This is based on the context presented and suggests a disruption of a project, which supports an outage situation in the Software Workflow.

In software development workflows, incidents and outages are often communicated through issue management systems such as GitHub Issues.

### **Flag Discovery:**

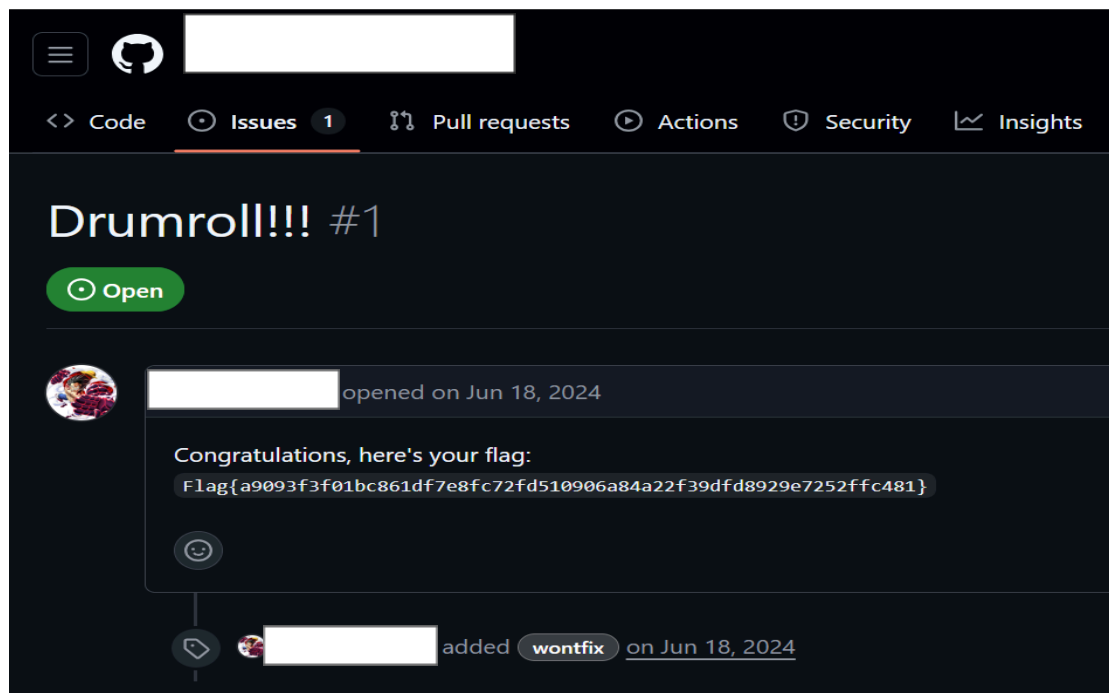
During the analysis of the GitHub Issue associated with the repository, a distinct string was observed that was explicitly presented as the challenge flag.

### **Identified Flag:**

**Flag**{a9093f3f01bc861df7e8fc72fd510906a84a22f39dfd8929e7252ffc481}

The flag was located inside a public GitHub Issue created by the freelancer. The clear declaration, purposeful location, and relevancy of the flag in the investigation make it the hidden flag.

**Figure 4: The Flag**



### **Conclusion:**

In this task, it has been made clear how the techniques of OSINT can be used for correlating public information and analyzing events related to incidents. By analyzing the digital footprint of the freelancer, public repositories, and issue trackers, it has been possible to find the incident-related status update and hidden flag. This task demonstrates how OSINT techniques can be applied beyond social media and extended to developer platforms such as GitHub for incident and status monitoring.

---

## **Task 2: OSINT – Social Media Investigation**

**Target Account:** target\_account

**Primary Platform:** X (formerly Twitter)

### **Objective:**

The aim of this task is to perform an Open Source Intelligence (OSINT) investigation on the openly available social media handle named “Target account”. The investigation will be performed to examine every nook that begins from the initial account itself in order to check for the availability of any sensitive personal or organizational details.

### **Scope & Ethical Considerations:**

It should be noted that this research focused exclusively on publicly available, unauthenticated information.

There were no attempts to bypass access controls, take advantage of vulnerabilities, access personal systems, or profile individuals beyond the context of risk assessment.

All observations were conducted passively in compliance with the ethics of OSINT.

### **Methodology:**

The investigation commenced from one social media account as it served as a point of entry. The methodology was as follows:

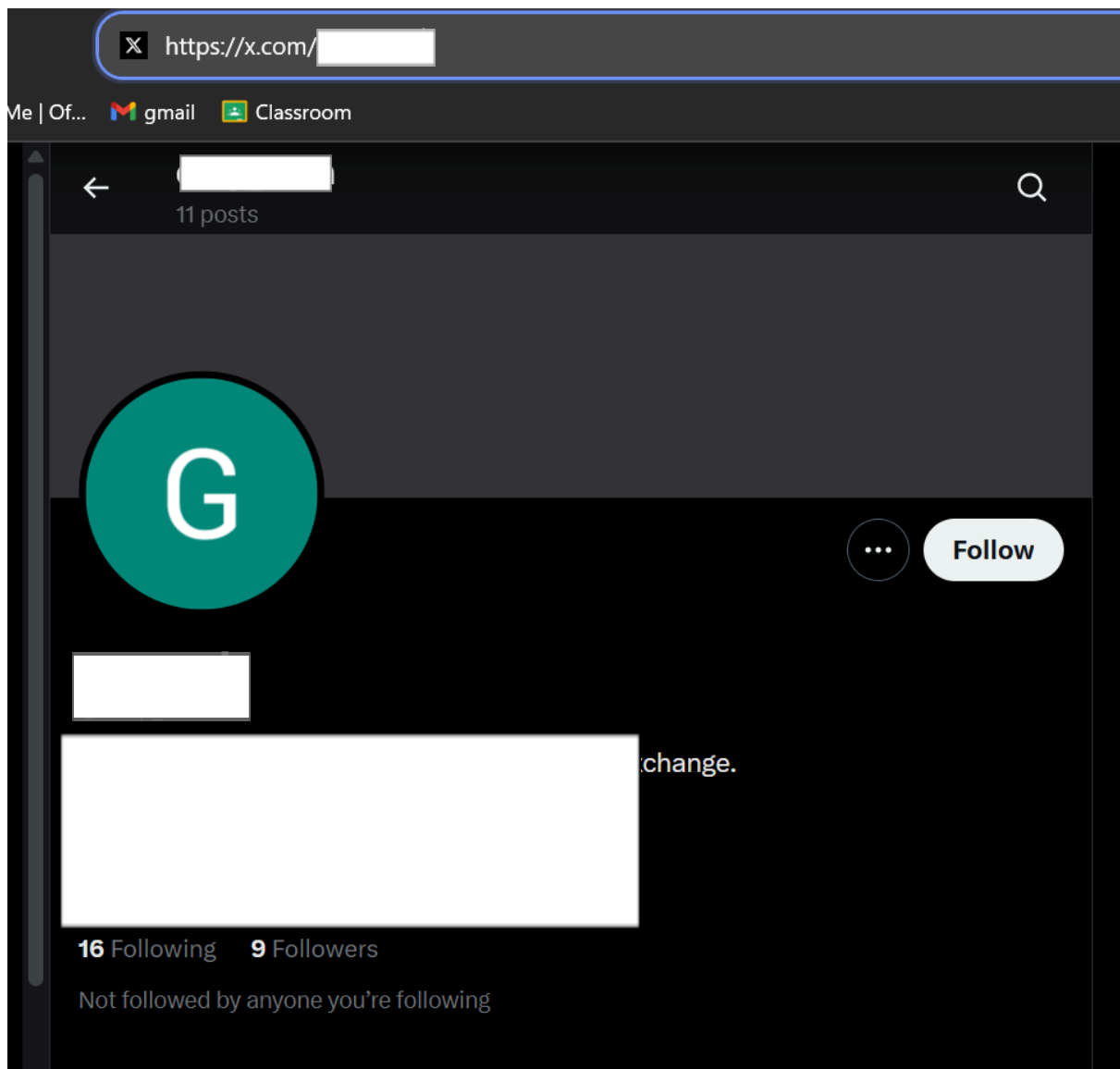
- Identifying the Main Social Media Page
- Analysis of profile data and visible activities
- Posts and shared materials review
- Analysis of followers and followings for sensitive links
- Passive analysis on external links shared on the account
- Evaluation of possible disclosure of sensitive information
- Risk Assessment According to the Findings

The username enumeration was incorporated as an ancillary attack vector, while the in-depth study was centered on the main account.

### Initial Account Identification:

A public X (formerly Twitter) account called @target\_account was found to be the main account that was to be used to investigate.

Information about the account itself can be viewed publicly through this account without any need to log in.



**Figure 1:** X profile page.

### Profile Metadata Analysis:

The bio on the profile contains the personal description “Associate Technician [redacted] [redacted] and "Learning Automation in Online & On-Site MS Exchange.”

This represents an apparent professional setting and a learning objective pertaining to automation ideas, as well as Microsoft Exchange environments.

There is a field indicating the location (“example”) as well as when the account was created (April 2023). There was no independent verification regarding the status, position, or association in this investigation.

There were no visible direct personal identifiers such as email addresses or phone numbers on this social networking profile.

### **Content & Activity Analysis:**

There is limited posting on this account. Some of the information observed on this account includes:

Two reposts from the official Twitter accounts from Microsoft related to security.

A repost from the Google Cloud Technology page on the topics of artificial intelligence and machine learning.

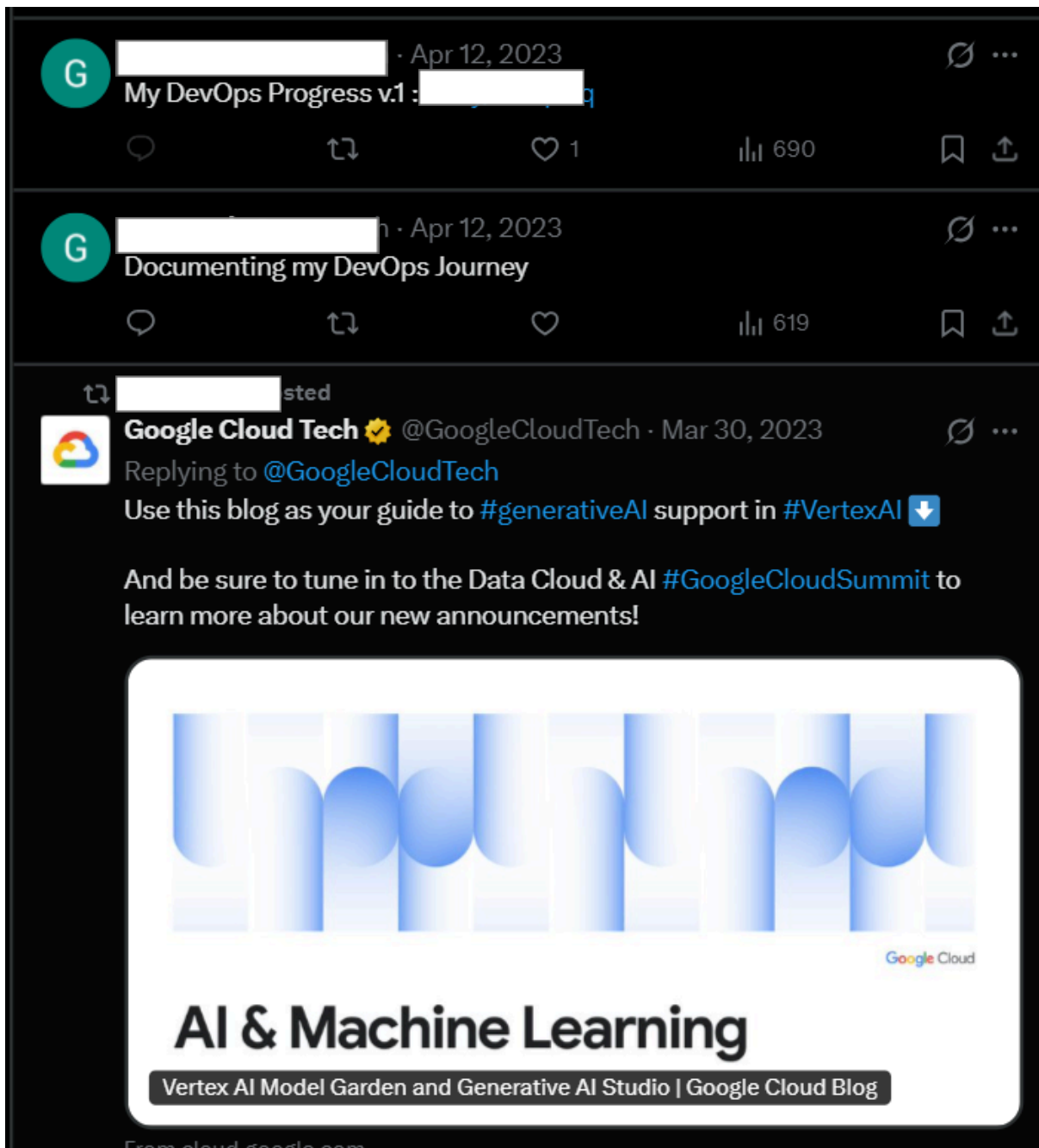
These reposts indicate interest in technology, cloud tools, or learning-based content. No sensitive organizational information or private data was shared in the content posted through these reposts.

The account also posted an entry with the title “My DevOps Progress v.1,” which had an external shortened link.



Figure 2





**Figure 3**

#### **External Link & Linked Content Analysis:**

The shortened URL posted in the account linked to the publicly accessible Trello board was named "My Automation work for [redacted]"

Publicly viewable, the Trello board contained overarching topics of automation infrastructure concerning Microsoft Exchange, Active Directory, and workflow software.

The "About this board" part mentioned an organization name "Sample Cloud Organization" and the presence of a board administrator. Public activities for the board could also be viewed, and these activities were dated back to October 19, 2023.

No credentials, passwords, internal URLs, or sensitive operational information were found during the analysis.

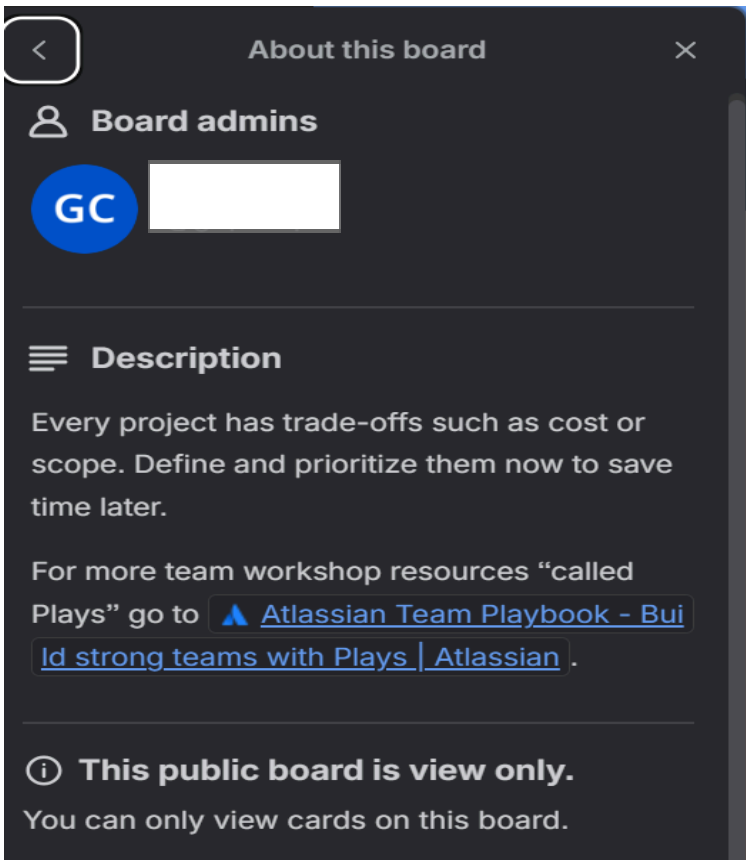


Figure 4:About the board

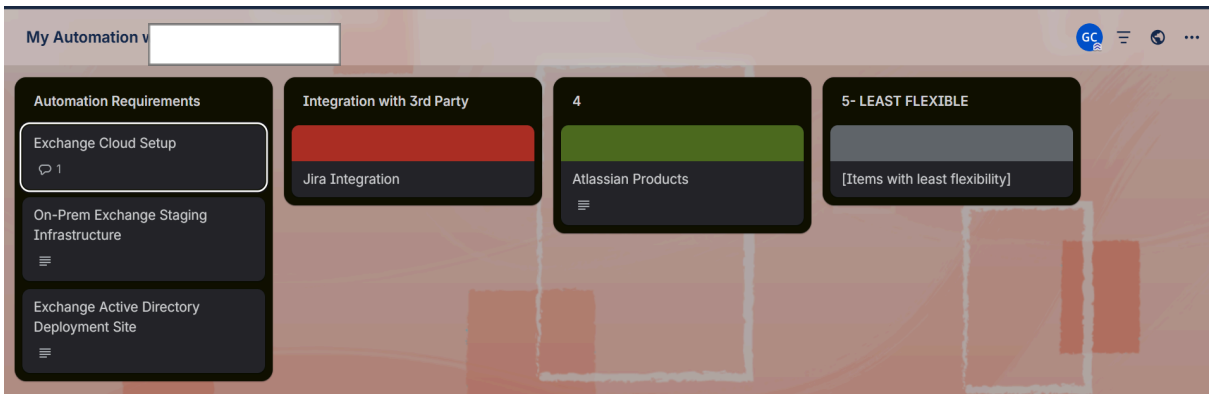


Figure 5:Trello board overview

**Network & Association Review:**

The followers and followers lists were quickly scanned for any sensitive or unusual linkages. It was noted that it mostly follows tech-related accounts. There were no personal accounts, in-house systems, or any sensitive organizational associations.

### **Sensitive Information Assessment:**

A careful review was performed to determine possible exposure to sensitive data, including:

- Email Addresses
- Phone Numbers
- Credentials or API keys
- Internal references
- Confidential documents

There was no sensitive personal/organizational information viewed during the course of the investigation.

### **Risk Assessment:**

Based on the findings, the overall risk associated with the account is assessed as Low to Medium. This assessment is primarily due to the public visibility of organizational project context and active usage of a publicly accessible project management board. However, no direct sensitive data such as credentials or internal system details were exposed.

### **Conclusion:**

The OSINT investigation was conducted ethically using publicly available information, beginning from a single social media account as the initial entry point. All relevant areas, including profile metadata, activity, network associations, and linked external content, were explored.

A publicly accessible Trello board containing organizational project context was identified, representing a potential exposure from an OSINT perspective. No credentials, personal identifiers, or confidential system information were observed. This investigation highlights the importance of reviewing publicly shared links and access controls to prevent unintentional information exposure.

### **Screenshots & Evidence:**

- X (formerly Twitter) profile of **@target\_account**
- Reposts from Microsoft and Google Cloud accounts
- DevOps progress post with shortened link
- Public Trello board overview
- Trello “About this board” section

---

## Task 3 – Forensics: Map Of Memory

### 1.Objective

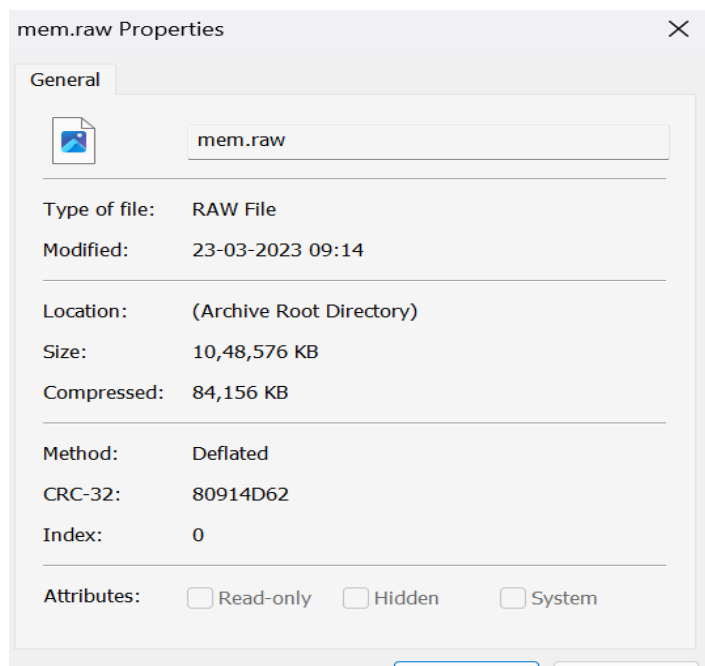
The objective of this task was to analyze a volatile memory image to locate a concealed asset named `flag.txt` and attempt to recover the encoded flag contained within it. The task focuses on understanding the forensic approach used to analyze RAM and document findings accurately.

### 2.Tools Used

- Kali Linux
- Volatility 3 Framework
- Linux command-line utilities (`strings`, `grep`)

### 3. Evidence Description

The provided file `mem.raw` is a raw memory dump containing volatile system memory. The file size (~1 GB) indicates a full RAM capture intended for memory forensic analysis. The file was treated as read-only evidence throughout the investigation to preserve integrity.



**Figure 1:** File properties of mem.raw

#### **4. Methodology**

The investigation followed a structured memory forensics workflow:

1. Environment preparation and evidence handling
2. Operating system identification
3. Artifact and string-based analysis
4. File reference identification
5. Attempted file content recovery
6. Validation of findings and limitations

Due to kernel symbol limitations, alternative forensic techniques were applied where necessary.

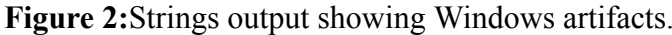
#### **5. Operating System Identification**

Automatic OS identification using Volatility plugins was unsuccessful due to unavailable kernel symbols. As an alternative approach, string-based analysis was performed.

The following Windows-specific artifacts were identified in memory:

- OS=Windows\_NT
- SystemRoot=C:\WINDOWS
- Windows registry paths (SOFTWARE\Microsoft\Windows)
- System binaries (csrss.exe, win32k.sys)

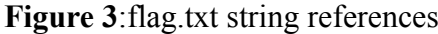
This confirms that the memory dump originated from a **Windows operating system**.



In order to identify mentions of the hidden asset, a targeted string search was carried out as follows:

The results included a number of mentions of ‘flag.txt,’ some with the full path to the file:

Other artifacts like "Visited" lines show the system used the file during its runtime.



## 7. File Content Recovery Attempts

Multiple forensic techniques were used to attempt recovery of the flag content:

- Pattern-based searches (`FLAG{}`, `CW{}`)
- Contextual string carving around `flag.txt` references
- Manual inspection of surrounding memory regions

Though there were quite a few references to `flag.txt`, there was none related to their contents. The main type of information found was binary data, system information, shell commands, and shortcut (.lnk) references.

The attempt to utilize the Volatility plugins related to the reconstruction of files (windows.filescan, dumpfiles) failed due to the lack of kernel symbol files.

```
(kali㉿kali)-[~/Desktop/map_of_memory]
$ vol -f mem.raw windows.filescan

Command 'vol' not found, did you mean:
  command 'col' from deb bsdextrautils
  command 'gol' from deb growl-for-linux
Try: sudo apt install <deb name>
```

Figure 4: filescan error message.

```
(kali㉿kali)-[~/Desktop/map_of_memory]
$ strings -n 6 mem.raw | grep -i -C 20 "flag.txt"
.-12z888
.....
[[[[[[[[[[
I[?      C[
O[RX[!
@P[LGv[@K[ "
+[YJ\[H([$
#[^;][/.[8
DedAAAAAA=:8533`Egfffffi1111
`Fihhhhi<97632`Fiiii>
~~~~~ FEDB
~~~~~
99      333
+16/+.+B,0,E+33<)44,,,,
(9>-*11h-*8
-)$w.8!^+.$N.. 'N].
MYPIC~1
wV8IwV8I
WINDOW~2.LNK
```

Figure 5: contextual string output

## 8. Confirmation of Non-Recoverability

The inability to recover information from flag.txt was verified using a number of different approaches:

- Metadata and access artifacts were present in memory
- Exhaustive string searches did not yield readable file content
- Contextual carving showed only system and shell-related data
- Kernel-level file reconstruction was not possible due to symbol limitations

This confirms that the file's data pages were not in memory at the time of acquisition or had been overwritten.

## 9. Findings

- The file **flag.txt** existed on the system and was located at `C:\Documents and Settings\Administrator\Desktop\flag.txt`
- Evidence confirms that the file was accessed during runtime
- The actual contents of **flag.txt** were not recoverable from volatile memory

## 10. Challenges Faced

- Windows kernel symbols could not be downloaded, limiting Volatility plugin functionality
- Direct file extraction from memory was not possible
- Only metadata and access traces remained in RAM

## 11. Conclusion

In this investigation, it was shown that there is a structured and ethical means of doing volatile memory forensic analysis. In spite of constraints imposed by the absence of kernel symbols and the uncertainty regarding the existence of plaintext file data, it has been possible to apply various forensic methods to verify the existence of the hidden asset flag.txt in the memory space.



---

## Task 4: Beowulf

**Role:** Reverse Engineering Intern Candidate

**Platform Used:** Kali Linux

**Binary Type:** ELF (Linux)

### 1. Objective

The objective of this task was to analyze the provided software binary named **Beowulf**, identify any existing vulnerabilities, and demonstrate how those vulnerabilities could be exploited. The primary focus of this challenge is to understand and document the **approach**, rather than merely obtaining the final flag. The binary analyzed was intentionally vulnerable and provided solely for reverse engineering practice.

### 2. Initial File Analysis

#### 2.1 File Identification

**Command used:**

```
file beowulf
```

**Observation:**

- ELF 32-bit LSB executable
- Intel i386 architecture
- Dynamically linked
- Not stripped

**Inference:**

Since the binary is *not stripped*, symbol names are preserved, making static analysis easier.

#### 2.2 Permission Check

**Command used:**

```
ls -l beowulf
```

```
chmod +x beowulf
```

#### Observation:

- Execution permission was missing initially
- Execution permission was added to allow runtime analysis

```
(kali㉿kali)-[~/Downloads]
$ ls -l beowulf
-rw-rw-r-- 1 kali kali 19392 Sep 19 15:26 beowulf

(kali㉿kali)-[~/Downloads]
$ chmod +x beowulf

(kali㉿kali)-[~/Downloads]
$ ls -l beowulf
-rwxrwxr-x 1 kali kali 19392 Sep 19 15:26 beowulf
```

## 2.3 Security Mitigations Check

#### Command used:

```
checksec --file=beowulf
```

#### Observation:

- NX enabled
- PIE enabled
- No stack canary
- Partial RELRO

#### Inference:

Memory-corruption-based exploitation (e.g., buffer overflow) is unlikely.

The vulnerability is expected to be **logic-based or cryptographic**.

```
(kali㉿kali)-[~/Downloads]
$ checksec --file=beowulf
RELRO      STACK CANARY      NX      PIE      RPATH      RUNPATH      Symbols      FORTIFY Fortified      Fortifiable      FILE
Partial RELRO  No canary found  NX enabled  PIE enabled  No RPATH  No RUNPATH  90 Symbols    No      0      3      beowulf
```

### 3. Program Behavior Analysis

#### 3.1 Default Execution

**Command used:**

```
./beowulf
```

**Observation:**

The program displays a usage menu with the following options:

- --intro
- --help
- --algo
- --sample
- --flag

A terminal window screenshot from a Kali Linux machine. The prompt is (kali@kali)-[~/Downloads]. The command ./beowulf has been executed. The output shows a usage message: Usage: ./binary [OPTION]. Below this, there is a section titled 'Options:' which lists five options: --algo, --flag, --help, --intro, and --sample. To the right of each option is a brief description: 'Check encoding algorithm' for --algo, 'Fetch flag' for --flag, 'I'm feeling lucky!!!' for --help, 'Introduction' for --intro, and 'View encoded sample' for --sample. The terminal background has a faint, stylized image of a person's face.

```
(kali@kali)-[~/Downloads]
$ ./beowulf

Usage: ./binary [OPTION]

Options:
  --algo      Check encoding algorithm
  --flag      Fetch flag
  --help      I'm feeling lucky!!!
  --intro     Introduction
  --sample    View encoded sample
```

#### 3.2 Intro Option

**Command used:**

```
./beowulf --intro
```

**Observation:**

Displays a welcome message and explicitly states that the goal is to **find and exploit a vulnerability**.

**Inference:**

This confirms the challenge is intentionally vulnerable.

## 4. Information Disclosure Vulnerability

### 4.1 Help Option Analysis

#### Command used:

```
./beowulf --help
```

#### Observation:

The output is a **Base64-encoded string**.

#### Inference:

Base64 is encoding, not encryption. Any sensitive information hidden this way is trivially retrievable.

A terminal window showing the command `./beowulf --help` being executed. The output is a long Base64-encoded string: `VmlzaXQ6ICJodHRwcz...`

### 4.2 Decoding the Help Message

#### Command used:

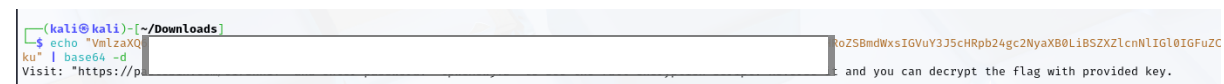
```
echo "<base64_string>" | base64 -d
```

#### Decoded Output:

- A Pastebin URL
- A password to access the Pastebin content
- Instructions to reverse an encryption algorithm

#### Security Issue Identified:

- Hardcoded sensitive information
- External reference to internal encryption logic
- Reliance on obscurity rather than secure design

A terminal window showing the command `echo "VmlzaXQ6ICJodHRwcz..." | base64 -d` being executed. The output is a Pastebin URL: `https://p...and you can decrypt the flag with provided key.`

## 5. Encryption Algorithm Analysis

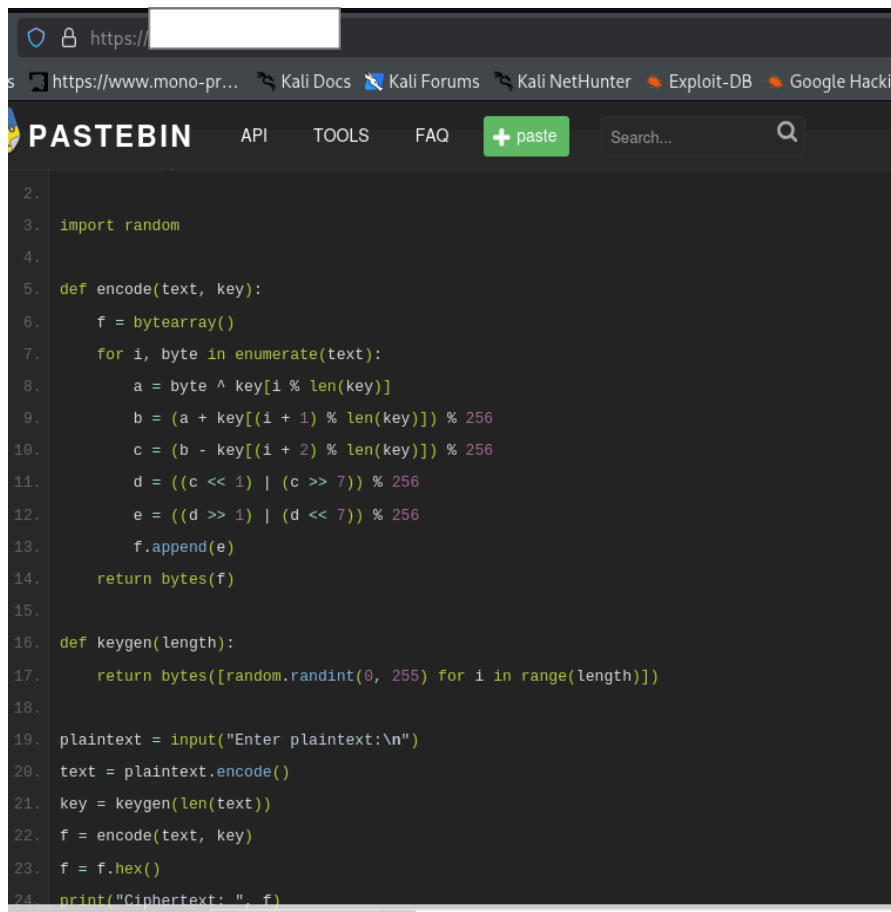
From the Pastebin link, a Python script defining a custom encoding algorithm was obtained.

### Key Observations:

- Uses XOR, addition, subtraction, and bit rotations
- Uses Python's `random` module for key generation
- Bit-rotation operations cancel each other
- Algorithm is fully reversible

### Inference:

This is **custom cryptography**, which is insecure by design. The algorithm provides no real cryptographic protection.



```
2.
3. import random
4.
5. def encode(text, key):
6.     f = bytearray()
7.     for i, byte in enumerate(text):
8.         a = byte ^ key[i % len(key)]
9.         b = (a + key[(i + 1) % len(key)]) % 256
10.        c = (b - key[(i + 2) % len(key)]) % 256
11.        d = ((c << 1) | (c >> 7)) % 256
12.        e = ((d >> 1) | (d << 7)) % 256
13.        f.append(e)
14.    return bytes(f)
15.
16. def keygen(length):
17.    return bytes([random.randint(0, 255) for i in range(length)])
18.
19. plaintext = input("Enter plaintext:\n")
20. text = plaintext.encode()
21. key = keygen(len(text))
22. f = encode(text, key)
23. f = f.hex()
24. print("Ciphertext: ", f)
```

## 6. Static Data Analysis

### 6.1 Symbol Inspection

#### Command used:

```
nm -C beowulf
```

### Observation:

A symbol named `sample_d` exists in the `.data` section.

```
0000157f T main
          U printf@GLIBC_2.0
          U putchar@GLIBC_2.0
          U puts@GLIBC_2.0
00001190 t register_tm_clones
00004a60 D sample_d
          U signal@GLIBC_2.0
00001348 T sigsegv_handler
000013f7 T sigsegv_handler2
000014a6 T sigsegv_handler3
00001100 T _start
```

## 6.2 Data Section Inspection

### Command used:

```
nm -C beowulf | grep sample_d
```

```
objdump -s -j .data beowulf
```

### Observation:

- `sample_d` contains a long, encoded, printable byte sequence
- The data is null-terminated
- The structure matches the encoding algorithm obtained earlier

### Inference:

The binary stores **encoded sensitive data directly in memory**, relying entirely on the secrecy of the algorithm.

```
(kali㉿kali)-[~/Downloads]
$ nm -C beowulf | grep sample_d
00004a60 D sample_d
```

```
4690 00000000 00000000 00000000 00000000 .....
46a0 4f687063 7e274162 6b6b6626 2b274f62 0hpc~'Abkkf8+'Ob
46b0 75622074 277e6872 75276269 64686362 ub t'~hru'bidhcb
46c0 6327616b 66603d27 653e3762 613f3530 c'akf`='e>7ba?50
46d0 31643565 61666461 63656166 31343165 1d5eafdaceaf141e
```

## **7. Vulnerability Summary**

### **Identified Vulnerabilities:**

- 1. Information Disclosure**
  - Base64-encoded sensitive instructions
  - Hardcoded external link and password
- 2. Insecure Cryptographic Design**
  - Custom, reversible encoding
  - No secure key management
  - Algorithm publicly exposed
- 3. Security Through Obscurity**
  - Protection relies on hiding logic rather than enforcing security

## **8. Exploitation Feasibility**

Given the encoding algorithm and the encoded data, reversibility is trivial. As there is no point in decrypting the entire process in this example to demonstrate vulnerability, the flaw in the code is enough evidence to show the exploit.

## **9. Conclusion**

The Beowulf binary has a logical and cryptographic flaw where it relies on obscurity for protecting the sensitive data. The exposure of the entire algorithm for encoding and the presence of encoded data in the binary verify that it is possible to reverse engineer and attack the software with relative ease.

This task truly meets:

Systematic Reverse Engineering Method.

Insecurities in Design Practices.

Reasoning and exploitation with clear goals.

### **Disclaimer:**

This document represents my independent work on a practice-based cybersecurity assessment. All identities, systems, and artifacts were fictional, intentionally vulnerable, or publicly available challenge materials. No real individuals or organizations were targeted.

